Module 1 : Foundations of Commodities Fa Value

Learning Objectives

By the end of this module, you will understand:

1. **What "fair value" means** in commodity markets and how it differs from other asset classes
2. **Economic foundations** including Kaldor's storage theory, Working's hypothesis, and conve yield
3. **Trading applications** - How to identify mispricing opportunities and manage risk
4. **The toolkit** - Introduction to the fair_value_toolkit library for practical implementations

---

Setup and Imports

In [1]:
```
import sys
sys.path.append('../..')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta

# Set plotting style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

# Import our fair value toolkit
from src.fair_value_toolkit import (
    PointInTimeDataFrame,
    FairValueModel,
    InventoryFairValueModel,
    CostOfCarryModel,
    SupplyDemandModel,
    CostCurveModel,
    EnsembleFairValueModel
)

from data.data_fetchers import create_sample_dataset

print("✓ Setup complete")
```

```
✓ Setup complete
```

Part 1 : What is "Fair Value" in Commodity Markets?

1.1 Defining Fair Value

Fair value in commodity markets represents an **theoretically justified price** based on fundame economic factors, distinct from the current market price which may be influenced by:

- **Supply and demand imbalances**
- **Storage costs and convenience yield**
- **Production costs (marginal cost curve)**
- **Macroeconomic factors** (interest rates, currency movements)
- **Market sentiment and speculative positioning**

The **gap between fair value and market price** creates trading opportunities:

```
Market Price > Fair Value  →  Overvalued  →  Short opportunity
Market Price < Fair Value  →  Undervalued →  Long opportunity
```

1.2 How Commodities Differ from Other Asset Classes

Stocks (Equity)

- **Valuation**: Discounted cash flows (dividends, earnings)
- **No expiration**: Can hold indefinitely
- **No storage cost**: Hold position at zero cost (ignoring financing)

Bonds (Fixed Income)

- **Valuation**: Present value of future cash flows
- **Fixed maturity**: Known end date
- **No physical delivery**: Settlement in cash

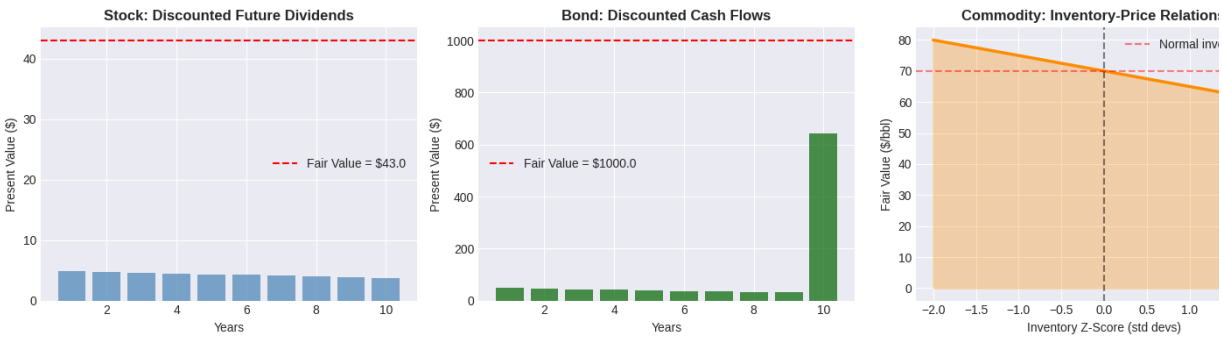
Commodities (Physical Goods)

- **Valuation**: Based on **physical supply/demand balance**
- **Storage costs**: Expensive to hold physical inventory
- **Convenience yield**: Value from immediate availability
- **Physical delivery**: Futures contracts can result in physical goods
- **Production economics**: Price tends toward marginal cost of production

Key Insight: Commodities have **intrinsic value** as physical goods needed for consumption or production. This creates unique valuation dynamics not present in financial assets.

In [2]:
```
# Visualization: Compare asset class value drivers

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Stock valuation
years = np.arange(1, 11)
dividends = 5 * (1.05 ** years)
```

```
discount_rate = 0.08
pv_dividends = dividends / ((1 + discount_rate) ** years)

axes[0].bar(years, pv_dividends, color='steelblue', alpha=0.7)
axes[0].set_title('Stock: Discounted Future Dividends', fontsize=12, font
axes[0].set_xlabel('Years')
axes[0].set_ylabel('Present Value ($)')
axes[0].axhline(y=pv_dividends.sum(), color='red', linestyle='--', label=
axes[0].legend()

# Bond valuation
coupon_dates = np.arange(1, 11)
coupons = np.ones(10) * 50  # $50 coupon
coupons[-1] += 1000  # Add principal at maturity
discount_rate_bond = 0.05
pv_coupons = coupons / ((1 + discount_rate_bond) ** coupon_dates)

axes[1].bar(coupon_dates, pv_coupons, color='darkgreen', alpha=0.7)
axes[1].set_title('Bond: Discounted Cash Flows', fontsize=12, fontweight=
axes[1].set_xlabel('Years')
axes[1].set_ylabel('Present Value ($)')
axes[1].axhline(y=pv_coupons.sum(), color='red', linestyle='--', label=f'
axes[1].legend()

# Commodity valuation
inventory_levels = np.linspace(-2, 2, 100)
prices = 70 - 5 * inventory_levels  # Price decreases with higher invento

axes[2].plot(inventory_levels, prices, linewidth=2.5, color='darkorange')
axes[2].fill_between(inventory_levels, prices, alpha=0.3, color='darkoran
axes[2].set_title('Commodity: Inventory-Price Relationship', fontsize=12,
axes[2].set_xlabel('Inventory Z-Score (std devs)')
axes[2].set_ylabel('Fair Value ($/bbl)')
axes[2].axvline(x=0, color='black', linestyle='--', alpha=0.5)
axes[2].axhline(y=70, color='red', linestyle='--', alpha=0.5, label='Norm
axes[2].legend()

plt.tight_layout()
plt.show()

print("The three asset classes have fundamentally different valuation fra
print("Commodities are unique in their dependence on physical supply/dema
```




```
The three asset classes have fundamentally different valuation frameworks.
Commodities are unique in their dependence on physical supply/demand balance.
```

1.3 The Role of Storage and Convenience Yield

Storage creates a fundamental link between spot and futures prices:

Cost of Carry Relationship:

$$ F_t(T) = S_t \cdot e^{(r + c - y)(T-t)} $$

Where:

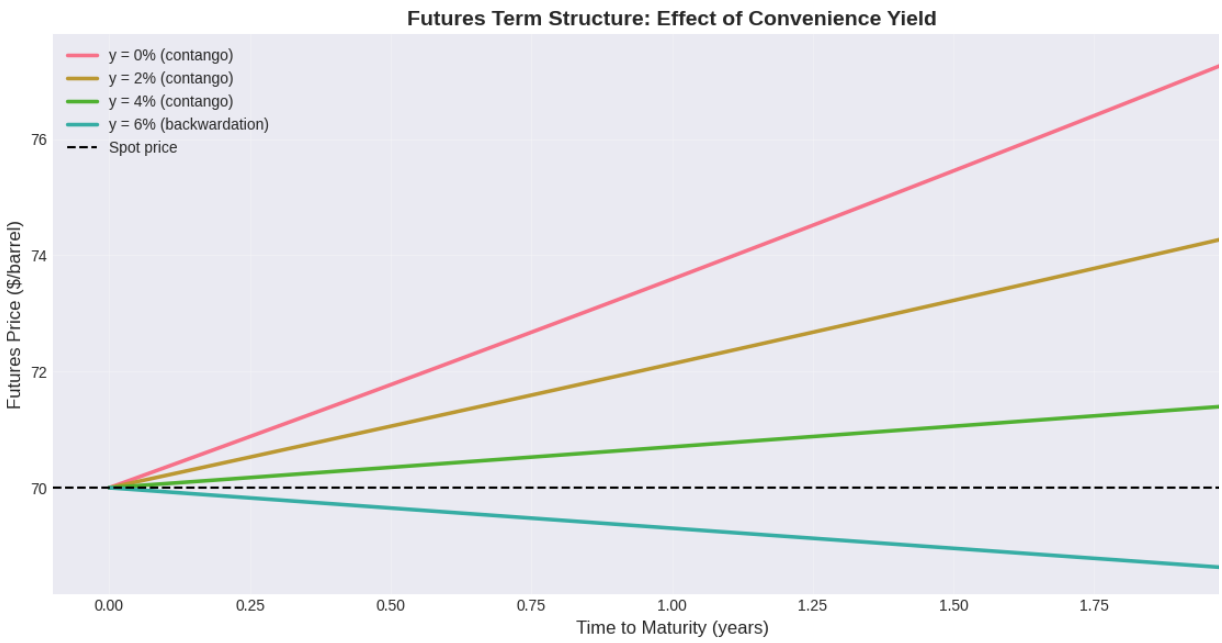
- $F_t(T)$ = Futures price at time t for delivery at T
- S_t = Spot price at time t
- r = Risk-free interest rate (financing cost)
- c = Physical storage cost rate
- y = Convenience yield
- $(T-t)$ = Time to maturity

Convenience Yield (y) represents the **benefit of holding physical inventory**:

- Ability to meet unexpected demand
- Avoid production disruptions
- Maintain customer relationships
- Flexibility in production scheduling

Key relationship: When inventories are low, convenience yield is high (storage is valuable).

In [3]:
```
# Demonstration: Cost of carry with varying convenience yield

spot_price = 70  # $/barrel
risk_free_rate = 0.03  # 3%
storage_cost = 0.02  # 2%
time_to_maturity = np.linspace(0, 2, 100)  # 0 to 2 years

# Calculate futures prices for different convenience yields
convenience_yields = [0.0, 0.02, 0.04, 0.06]

fig, ax = plt.subplots(figsize=(12, 6))

for cy in convenience_yields:
    futures_prices = spot_price * np.exp((risk_free_rate + storage_cost -
    label = f'y = {cy*100:.0f}% (' + ('backwardation' if cy > risk_free_r
    ax.plot(time_to_maturity, futures_prices, linewidth=2.5, label=label)

ax.axhline(y=spot_price, color='black', linestyle='--', linewidth=1.5, la
ax.set_xlabel('Time to Maturity (years)', fontsize=12)
ax.set_ylabel('Futures Price ($/barrel)', fontsize=12)
ax.set_title('Futures Term Structure: Effect of Convenience Yield', fonts
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nKey Observations:")
print("1. When y < (r + c): Contango (futures > spot) - Normal storage ec
print("2. When y > (r + c): Backwardation (futures < spot) - High conveni
print("3. High convenience yield typically occurs when inventories are ve
```




```
Key Observations:
1. When y < (r + c): Contango (futures > spot) - Normal storage economics
2. When y > (r + c): Backwardation (futures < spot) - High convenience yield
3. High convenience yield typically occurs when inventories are very low
```

Part 2: Economic Foundations

2.1 Kaldor's Storage Theory (1939)

Nicholas Kaldor first formalized the concept of convenience yield:

Key Propositions:

1. **Storage is an economic activity** - Inventory holders must be compensated
2. **Futures-spot spread reflects storage costs** - Net carrying costs determine term structure
3. **Convenience yield is the invisible benefit** - Value from holding physical stock

Equation: $$ F - S = (r + c) \cdot S - Y $$

Where Y is the total convenience yield (not a rate).

Implication for Fair Value: When futures trade at a premium (contango), it suggests high invent
and low convenience yield. When futures trade at a discount (backwardation), inventories are lik

2.2 Working's Hypothesis (1949)

Holbrook Working extended Kaldor's theory with empirical observations:

Core Finding: The **marginal convenience yield decreases with inventory levels**

$$ \frac{\partial y}{\partial I} < 0 $$

Working Curve: Non-linear relationship between inventory and convenience yield:

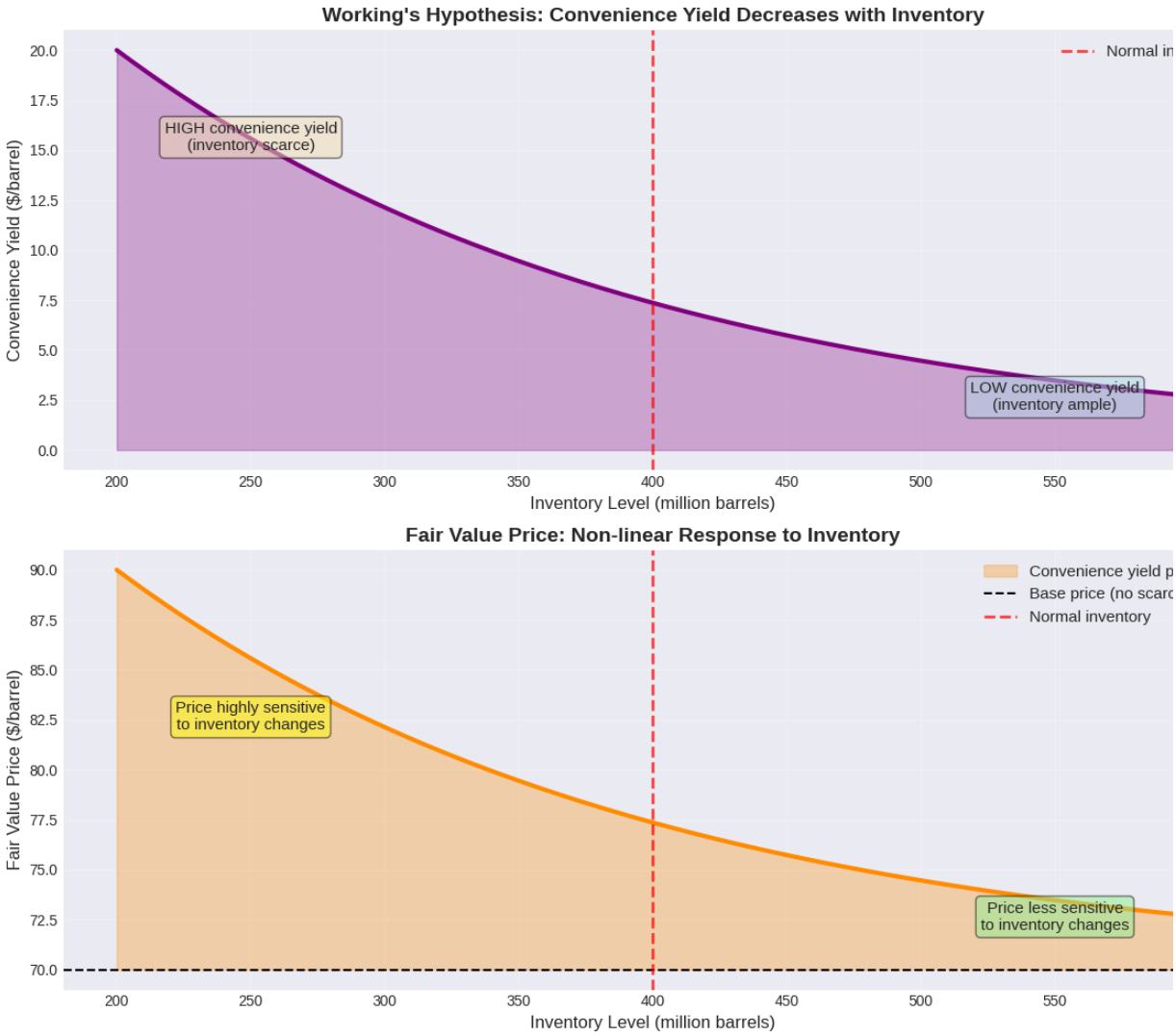
- **Very low inventory**: Convenience yield is extremely high (steep curve)
- **Moderate inventory**: Convenience yield declines gradually
- **High inventory**: Convenience yield approaches zero (flat curve)

Trading Implication:

- Price is most sensitive to inventory changes when stocks are low
- Price is less sensitive when stocks are ample

In [4]:
```
# Visualization: Working's Hypothesis - Non-linear inventory-price relati

# Create inventory levels (in millions of barrels)
inventory = np.linspace(200, 600, 200)
inventory_normal = 400  # Normal inventory level

# Working's convenience yield function (non-linear)
# Convenience yield is high when inventory is low
convenience_yield = 20 * np.exp(-0.005 * (inventory - 200))

# Price impact from convenience yield
# Base price is $70, convenience yield adds to price when inventory is lo
base_price = 70
price = base_price + convenience_yield

fig, axes = plt.subplots(2, 1, figsize=(12, 10))

# Panel 1: Convenience yield vs inventory
axes[0].plot(inventory, convenience_yield, linewidth=3, color='purple')
axes[0].fill_between(inventory, convenience_yield, alpha=0.3, color='purp
axes[0].axvline(x=inventory_normal, color='red', linestyle='--', linewidt
axes[0].set_xlabel('Inventory Level (million barrels)', fontsize=12)
axes[0].set_ylabel('Convenience Yield ($/barrel)', fontsize=12)
axes[0].set_title("Working's Hypothesis: Convenience Yield Decreases with
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)

# Add annotations
axes[0].annotate('HIGH convenience yield\n(inventory scarce)',
                xy=(250, 15), fontsize=11, ha='center',
                bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5)
axes[0].annotate('LOW convenience yield\n(inventory ample)',
                xy=(550, 2), fontsize=11, ha='center',
                bbox=dict(boxstyle='round', facecolor='lightblue', alpha=

# Panel 2: Fair value price vs inventory
axes[1].plot(inventory, price, linewidth=3, color='darkorange')
axes[1].fill_between(inventory, base_price, price, alpha=0.3, color='dark
axes[1].axhline(y=base_price, color='black', linestyle='--', linewidth=1.
axes[1].axvline(x=inventory_normal, color='red', linestyle='--', linewidt
axes[1].set_xlabel('Inventory Level (million barrels)', fontsize=12)
axes[1].set_ylabel('Fair Value Price ($/barrel)', fontsize=12)
axes[1].set_title('Fair Value Price: Non-linear Response to Inventory', f
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)

# Add price sensitivity annotation
```

```
axes[1].annotate('Price highly sensitive\nto inventory changes',
                 xy=(250, 82), fontsize=11, ha='center',
                 bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.5
axes[1].annotate('Price less sensitive\nto inventory changes',
                 xy=(550, 72), fontsize=11, ha='center',
                 bbox=dict(boxstyle='round', facecolor='lightgreen', alpha

plt.tight_layout()
plt.show()

print("\nWorking's Key Insight:")
print("The relationship between inventory and price is NON-LINEAR.")
print("This creates asymmetric trading opportunities - inventory shocks m
```




```
Working's Key Insight:
The relationship between inventory and price is NON-LINEAR.
This creates asymmetric trading opportunities - inventory shocks matter more
stocks are low!
```

2.3 Backwardation vs Contango

The **futures term structure** provides signals about market conditions:

Contango (Normal Market)

- **Futures > Spot** ($F_t > S_t$)
- **Low convenience yield** (y < r + c)

- **High inventories**
- Storage costs dominate
- Upward-sloping term structure

Backwardation (Tight Market)

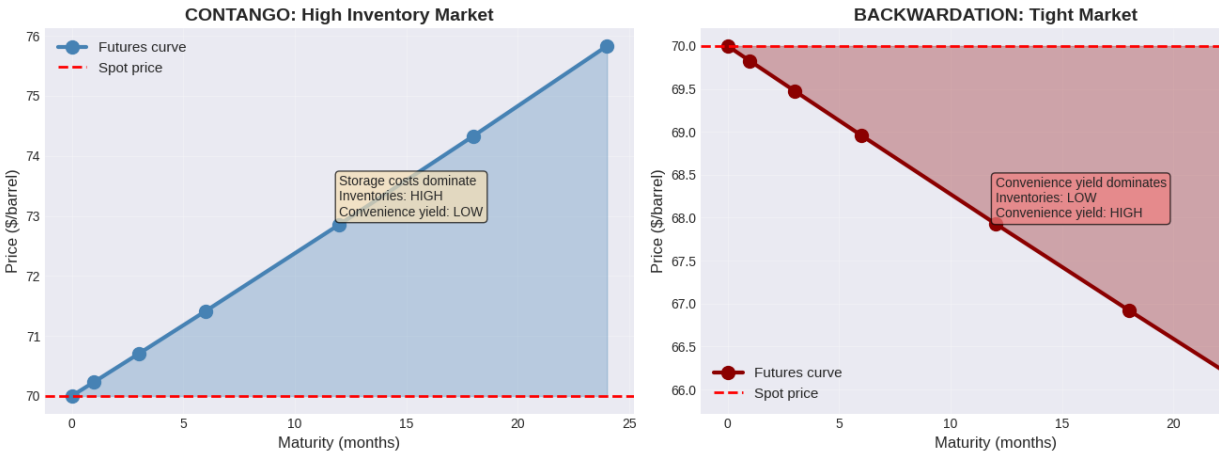
- **Spot > Futures** ($S_t > F_t$)
- **High convenience yield** (y > r + c)
- **Low inventories**
- Immediate demand is urgent
- Downward-sloping term structure

Trading Strategy:

- Backwardation suggests market tightness → Bullish signal
- Extreme contango suggests oversupply → Bearish signal

In [5]:
```
# Generate sample term structures for contango and backwardation

maturities = np.array([0, 1, 3, 6, 12, 18, 24])  # months
spot_price = 70

# Contango scenario: High inventory, low convenience yield
contango_prices = spot_price * np.exp((0.03 + 0.02 - 0.01) * maturities /

# Backwardation scenario: Low inventory, high convenience yield
backwardation_prices = spot_price * np.exp((0.03 + 0.02 - 0.08) * maturit

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Contango
axes[0].plot(maturities, contango_prices, 'o-', linewidth=3, markersize=1
axes[0].axhline(y=spot_price, color='red', linestyle='--', linewidth=2, l
axes[0].fill_between(maturities, spot_price, contango_prices, alpha=0.3,
axes[0].set_xlabel('Maturity (months)', fontsize=12)
axes[0].set_ylabel('Price ($/barrel)', fontsize=12)
axes[0].set_title('CONTANGO: High Inventory Market', fontsize=14, fontwei
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)
axes[0].text(12, 73, 'Storage costs dominate\nInventories: HIGH\nConvenie
             fontsize=10, bbox=dict(boxstyle='round', facecolor='wheat', a

# Backwardation
axes[1].plot(maturities, backwardation_prices, 'o-', linewidth=3, markers
axes[1].axhline(y=spot_price, color='red', linestyle='--', linewidth=2, l
axes[1].fill_between(maturities, backwardation_prices, spot_price, alpha=
axes[1].set_xlabel('Maturity (months)', fontsize=12)
axes[1].set_ylabel('Price ($/barrel)', fontsize=12)
axes[1].set_title('BACKWARDATION: Tight Market', fontsize=14, fontweight=
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)
axes[1].text(12, 68, 'Convenience yield dominates\nInventories: LOW\nConv
             fontsize=10, bbox=dict(boxstyle='round', facecolor='lightcora

plt.tight_layout()
plt.show()
```

```
print("\nTrading Signals:")
print("• Contango → Ample supply → Consider short positions or wait for b
print("• Backwardation → Tight market → Consider long positions or spread
print("• Extreme term structure shapes often mean-revert as supply/demand
```




```
Trading Signals:
• Contango → Ample supply → Consider short positions or wait for better entry
• Backwardation → Tight market → Consider long positions or spread trades
• Extreme term structure shapes often mean-revert as supply/demand rebalances
```

Part 3 : Why Fair Value Models Matter for Trading

3.1 Identifying Mispricing Opportunities

Fair value models create a **systematic framework** for identifying when prices deviate from fundamentals:

Mispricing Signal: $$ \text{Mispricing} = \frac{\text{Market Price} - \text{Fair Value}}{\text{Fair V
\times 100 \% $$

Trading Rules:

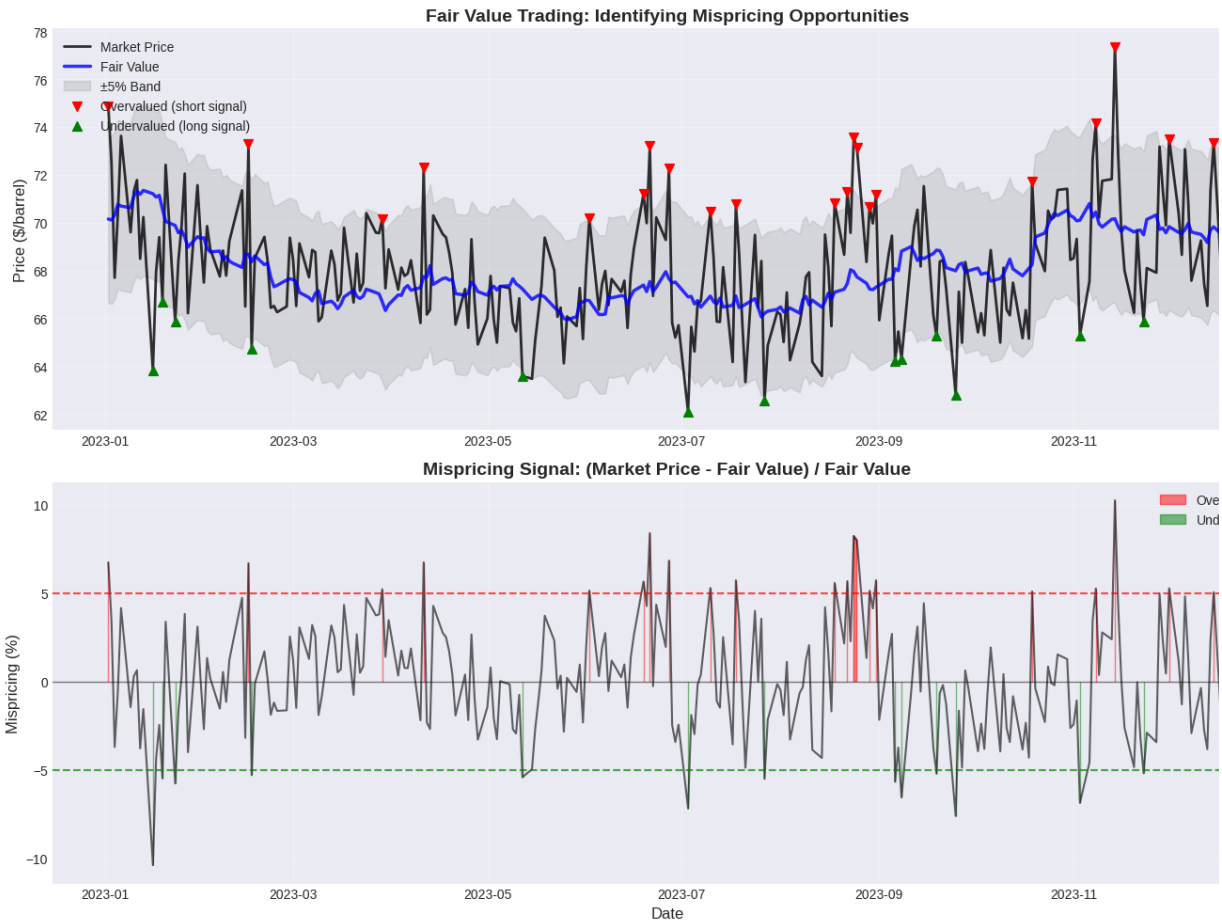
- **Overvalued** (Mispricing > + 5 %): Consider short positions
- **Fairly valued** (|Mispricing| ≤ 5 %): Hold or avoid
- **Undervalued** (Mispricing < - 5 %): Consider long positions

Why models matter:

1. **Discipline**: Removes emotion from trading decisions
2. **Repeatability**: Creates a systematic process
3. **Measurability**: Can backtest and evaluate performance
4. **Risk management**: Quantifies uncertainty via confidence intervals

In [6]:
```
# Simulate a time series showing mispricing opportunities

np.random.seed(42)
dates = pd.date_range('2023-01-01', periods=252, freq='B')

# Generate fair value (mean-reverting around $70)
```

```
fair_value = 70 + np.cumsum(np.random.randn(252) * 0.3)

# Generate market price (fair value + noise + momentum)
noise = np.random.randn(252) * 2
momentum = np.convolve(noise, np.ones(10)/10, mode='same')  # Add momentu
market_price = fair_value + noise + momentum

# Calculate mispricing
mispricing = (market_price - fair_value) / fair_value * 100

fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Panel 1: Price vs Fair Value
axes[0].plot(dates, market_price, linewidth=2, color='black', label='Mark
axes[0].plot(dates, fair_value, linewidth=2.5, color='blue', label='Fair
axes[0].fill_between(dates, fair_value * 0.95, fair_value * 1.05, alpha=0

# Highlight trading opportunities
overvalued = mispricing > 5
undervalued = mispricing < -5

axes[0].scatter(dates[overvalued], market_price[overvalued], color='red',
               label='Overvalued (short signal)', zorder=5)
axes[0].scatter(dates[undervalued], market_price[undervalued], color='gre
               label='Undervalued (long signal)', zorder=5)

axes[0].set_ylabel('Price ($/barrel)', fontsize=12)
axes[0].set_title('Fair Value Trading: Identifying Mispricing Opportuniti
axes[0].legend(fontsize=10, loc='upper left')
axes[0].grid(True, alpha=0.3)

# Panel 2: Mispricing percentage
axes[1].fill_between(dates, 0, mispricing, where=(mispricing > 5), color=
axes[1].fill_between(dates, 0, mispricing, where=(mispricing < -5), color
axes[1].plot(dates, mispricing, linewidth=1.5, color='black', alpha=0.6)
axes[1].axhline(y=5, color='red', linestyle='--', linewidth=1.5, alpha=0.
axes[1].axhline(y=-5, color='green', linestyle='--', linewidth=1.5, alpha
axes[1].axhline(y=0, color='black', linestyle='-', linewidth=1, alpha=0.5

axes[1].set_xlabel('Date', fontsize=12)
axes[1].set_ylabel('Mispricing (%)', fontsize=12)
axes[1].set_title('Mispricing Signal: (Market Price - Fair Value) / Fair
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Calculate statistics
n_overvalued = overvalued.sum()
n_undervalued = undervalued.sum()
avg_mispricing = np.abs(mispricing).mean()

print(f"\nTrading Opportunity Summary:")
print(f"• Overvalued signals: {n_overvalued} days ({n_overvalued/252*100:
print(f"• Undervalued signals: {n_undervalued} days ({n_undervalued/252*1
print(f"• Average absolute mispricing: {avg_mispricing:.2f}%")
print(f"• Total trading opportunities: {n_overvalued + n_undervalued} day
```




```
Trading Opportunity Summary:
• Overvalued signals: 21 days (8.3%)
• Undervalued signals: 13 days (5.2%)
• Average absolute mispricing: 2.64%
• Total trading opportunities: 34 days
```

3.2 Risk Management Applications

Fair value models provide **quantified uncertainty** through confidence intervals:

95% Confidence Interval: $$ \text{CI}_{95\%} = \text{Fair Value} \pm 1.96 \times \sigma_{\text{residual}} $$

Risk Management Uses:

1. **Stop Loss Placement**: Exit if price moves beyond confidence interval
2. **Position Sizing**: Smaller positions when uncertainty is high
3. **Entry Timing**: Wait for price to deviate beyond 1 standard deviation
4. **Portfolio Construction**: Allocate capital based on conviction (width of CI)

3.3 Position Sizing with Uncertainty

Kelly Criterion for optimal position sizing:

$$ f^* = \frac{p \cdot b - (1-p)}{b} $$

Where:

- f^* = Fraction of capital to risk

- p = Probability of winning
- b = Odds (win/loss ratio)

Fair value models help estimate p:

- When mispricing > 2σ: $p \approx 0.75$ (high probability mean reversion)
- When mispricing > 1σ: $p \approx 0.60$ (moderate probability)
- When mispricing < 1σ: $p \approx 0.50$ (coin flip)

Rule of thumb: Position size should scale with:

1. Magnitude of mispricing (larger deviation = larger size)
2. Confidence in fair value (narrower CI = larger size)
3. Recent model accuracy (better track record = larger size)

In [7]:
```
# Position sizing example based on mispricing magnitude and confidence

# Scenario parameters
account_size = 1_000_000  # $1M account
risk_per_trade_base = 0.02  # Base 2% risk per trade

# Create scenarios with different mispricing and confidence levels
scenarios = pd.DataFrame({
    'scenario': ['Weak Signal', 'Moderate Signal', 'Strong Signal', 'Very
    'mispricing_pct': [3, 6, 10, 15],
    'confidence_interval_width': [8, 6, 4, 3],  # % of fair value
    'fair_value': [70, 70, 70, 70],
    'market_price': [67.9, 65.8, 63.0, 59.5]
})

# Calculate position sizing multipliers
# Higher mispricing = higher multiplier
scenarios['mispricing_multiplier'] = (scenarios['mispricing_pct'] / 5).cl

# Narrower confidence interval = higher multiplier
scenarios['confidence_multiplier'] = (8 / scenarios['confidence_interval_

# Combined multiplier
scenarios['combined_multiplier'] = scenarios['mispricing_multiplier'] * s

# Calculate position size
scenarios['risk_pct'] = (risk_per_trade_base * scenarios['combined_multip
scenarios['position_size'] = account_size * scenarios['risk_pct']

# Calculate number of contracts (assume each contract = $1000 notional)
scenarios['n_contracts'] = (scenarios['position_size'] / 1000).astype(int

# Display
print("\n" + "="*80)
print("POSITION SIZING BASED ON FAIR VALUE CONVICTION")
print("="*80)
print(f"\nAccount Size: ${account_size:,.0f}")
print(f"Base Risk Per Trade: {risk_per_trade_base*100:.1f}%\n")

for idx, row in scenarios.iterrows():
    print(f"\n{row['scenario'].upper()}:")
    print(f"  Fair Value: ${row['fair_value']:.2f}")
```

```
    print(f"  Market Price: ${row['market_price']:.2f}")
    print(f"  Mispricing: {row['mispricing_pct']:.1f}%")
    print(f"  Confidence Interval Width: ±{row['confidence_interval_width
    print(f"  → Position Size: ${row['position_size']:,.0f} ({row['risk_p
    print(f"  → Number of Contracts: {row['n_contracts']}")

# Visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Bar chart of position sizes
colors = ['lightblue', 'skyblue', 'orange', 'red']
axes[0].bar(scenarios['scenario'], scenarios['position_size'], color=colo
axes[0].set_ylabel('Position Size ($)', fontsize=12)
axes[0].set_title('Position Sizing by Signal Strength', fontsize=14, font
axes[0].tick_params(axis='x', rotation=15)
axes[0].grid(axis='y', alpha=0.3)

# Add value labels
for i, (idx, row) in enumerate(scenarios.iterrows()):
    axes[0].text(i, row['position_size'] + 5000, f"${row['position_size']
                 ha='center', fontsize=10, fontweight='bold')

# Scatter plot of risk vs return potential
axes[1].scatter(scenarios['mispricing_pct'], scenarios['risk_pct']*100,
                s=scenarios['position_size']/1000, c=colors, alpha=0.6, ed

for idx, row in scenarios.iterrows():
    axes[1].annotate(row['scenario'],
                     xy=(row['mispricing_pct'], row['risk_pct']*100),
                     xytext=(10, 10), textcoords='offset points',
                     fontsize=9, bbox=dict(boxstyle='round', facecolor='wh

axes[1].set_xlabel('Mispricing (%)', fontsize=12)
axes[1].set_ylabel('Risk Allocated (% of account)', fontsize=12)
axes[1].set_title('Risk vs Opportunity (bubble size = position size)', fo
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n" + "="*80)
print("KEY TAKEAWAY: Scale position size with both mispricing magnitude a
print("="*80)
```

```
================================================================================
POSITION SIZING BASED ON FAIR VALUE CONVICTION
================================================================================

Account Size: $1,000,000
Base Risk Per Trade: 2.0%


WEAK SIGNAL:
  Fair Value: $70.00
  Market Price: $67.90
  Mispricing: 3.0%
  Confidence Interval Width: ±8.0%
  → Position Size: $12,000 (1.2% of account)
  → Number of Contracts: 12

MODERATE SIGNAL:
  Fair Value: $70.00
  Market Price: $65.80
  Mispricing: 6.0%
  Confidence Interval Width: ±6.0%
  → Position Size: $32,000 (3.2% of account)
  → Number of Contracts: 32

STRONG SIGNAL:
  Fair Value: $70.00
  Market Price: $63.00
  Mispricing: 10.0%
  Confidence Interval Width: ±4.0%
  → Position Size: $80,000 (8.0% of account)
  → Number of Contracts: 80

VERY STRONG SIGNAL:
  Fair Value: $70.00
  Market Price: $59.50
  Mispricing: 15.0%
  Confidence Interval Width: ±3.0%
  → Position Size: $100,000 (10.0% of account)
  → Number of Contracts: 100
```

```
================================================================================
KEY TAKEAWAY: Scale position size with both mispricing magnitude and confiden
================================================================================
```

Part 4: Course Toolkit Introduction

4.1 Overview of fair_value_toolkit Library

The `fair_value_toolkit` provides production-ready implementations of:

Core Components:

1. **Point-in-Time Data Management**
   - `PointInTimeRecord`: Single data observation with temporal metadata
   - `PointInTimeDataFrame`: DataFrame with as-of query capabilities
   - `PointInTimeDatabase`: SQLite-based temporal data store
2. **Fair Value Models**
   - `InventoryFairValueModel`: Based on storage theory
   - `CostOfCarryModel`: Based on futures-spot arbitrage
   - `SupplyDemandModel`: Based on fundamental balance
   - `CostCurveModel`: Based on marginal production cost
   - `EnsembleFairValueModel`: Combines multiple models
3. **Validation and Backtesting**
   - `WalkForwardValidator`: Temporal cross-validation
   - `detect_data_leakage`: Ensures no lookahead bias
   - `calculate_model_decay`: Tracks model degradation
4. **Signal Generation**
   - `SignalGenerator`: Convert fair value to trading signals
   - `PositionSizer`: Kelly criterion-based sizing
   - `RiskManager`: Portfolio-level risk controls

4.2 Demo: PointInTimeDataFrame

The `PointInTimeDataFrame` solves the **look-ahead bias problem** in backtesting.

Problem: Economic data is released with a lag. Using final revised data creates unrealistic back results.

Solution: Store publication dates and query data "as of" historical dates.

In [8]:
```
# Demo: Point-in-time data with publication lags

from src.fair_value_toolkit.point_in_time import create_sample_pit_data

# Create sample data with publication lags and revisions
pit_data = create_sample_pit_data(
    series_id='CRUDE_INVENTORY',
    start_date='2023-01-01',
    end_date='2024-01-01',
    frequency='W-FRI',  # Weekly Friday observations
```

```
    publication_lag_days=4,  # Published 4 days after observation
    revision_probability=0.3,  # 30% of observations get revised
    revision_magnitude=0.02  # Revisions are ~2% of value
)

print(pit_data)
print(f"\nTotal records: {len(pit_data)}")
print(f"Unique observations: {pit_data.df['observation_date'].nunique()}"

# Publication lag statistics
lag_stats = pit_data.publication_lag_stats()
print("\nPublication Lag Statistics:")
for key, value in lag_stats.items():
    print(f"  {key}: {value:.1f}")
```

```
PointInTimeDataFrame(series_id=CRUDE_INVENTORY, records=67, observations=52)

Total records: 67
Unique observations: 52

Publication Lag Statistics:
  mean_lag_days: 5.6
  median_lag_days: 4.0
  min_lag_days: 4.0
  max_lag_days: 11.0
  std_lag_days: 2.9
```

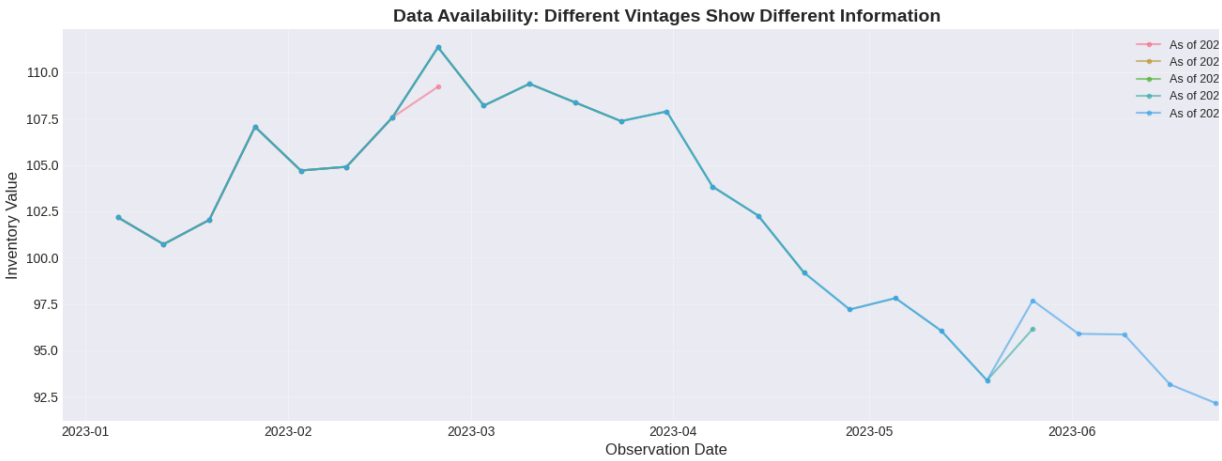
In [9]:

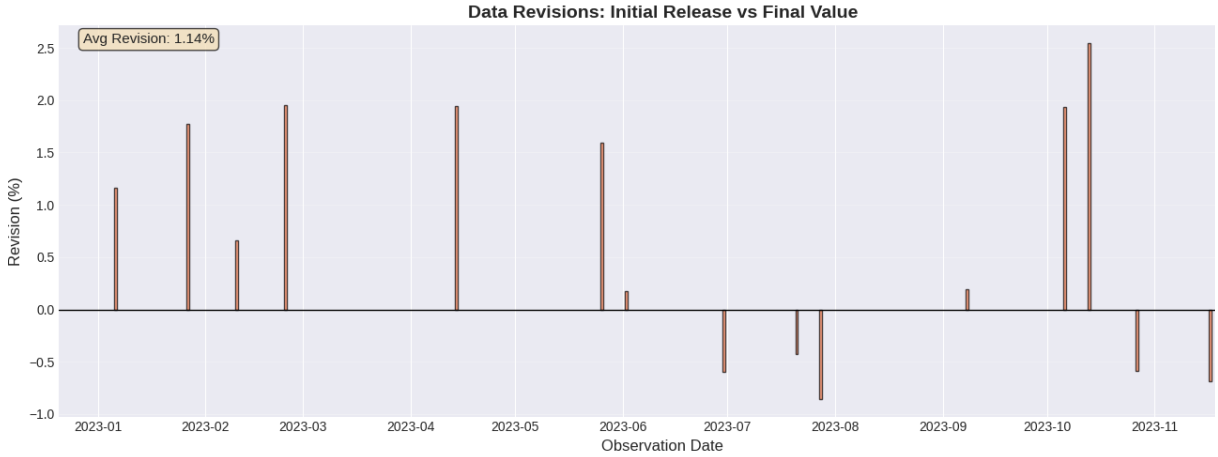
```
# Query data "as of" a historical date

query_date = datetime(2023, 6, 15)

# Get data as it was known on June 15, 2023
known_data = pit_data.as_of(query_date)

print(f"\nData known as of {query_date.date()}:")
print(f"Number of observations available: {len(known_data)}")
print(f"\nLast 5 observations:")
print(known_data[['value', 'publication_date', 'revision']].tail())

# Compare to final data (with all revisions)
final_date = datetime(2024, 2, 1)
final_data = pit_data.as_of(final_date)

print(f"\nData as of {final_date.date()} (includes all revisions):")
print(f"Number of observations: {len(final_data)}")
```

```
Data known as of 2023-06-15:
Number of observations available: 23

Last 5 observations:
                      value    publication_date  revision
observation_date
2023-05-12        96.052684 2023-05-16 10:30:00         0
2023-05-19        93.377532 2023-05-23 10:30:00         0
2023-05-26        97.685231 2023-06-06 10:30:00         1
2023-06-02        95.888652 2023-06-13 10:30:00         1
2023-06-09        95.851371 2023-06-13 10:30:00         0

Data as of 2024-02-01 (includes all revisions):
Number of observations: 52
```

In [10]:

```
# Visualization: Effect of publication lags and revisions

# Get data at different vintages
vintage_dates = pd.date_range('2023-03-01', '2023-07-01', freq='MS')

fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Panel 1: Data availability over time
for vintage in vintage_dates:
    vintage_data = pit_data.as_of(vintage)
    axes[0].plot(vintage_data.index, vintage_data['value'],
                 marker='o', markersize=3, alpha=0.6,
                 label=f"As of {vintage.strftime('%Y-%m-%d')}")

axes[0].set_xlabel('Observation Date', fontsize=12)
axes[0].set_ylabel('Inventory Value', fontsize=12)
axes[0].set_title('Data Availability: Different Vintages Show Different I
                  fontsize=14, fontweight='bold')
axes[0].legend(fontsize=9)
axes[0].grid(True, alpha=0.3)

# Panel 2: Revision impact
rev_stats = pit_data.revision_statistics()

if not rev_stats.empty:
    axes[1].bar(rev_stats['observation_date'], rev_stats['revision_pct'],
                color='coral', alpha=0.7, edgecolor='black')
    axes[1].axhline(y=0, color='black', linestyle='-', linewidth=1)
    axes[1].set_xlabel('Observation Date', fontsize=12)
    axes[1].set_ylabel('Revision (%)', fontsize=12)
    axes[1].set_title('Data Revisions: Initial Release vs Final Value',
                      fontsize=14, fontweight='bold')
    axes[1].grid(True, alpha=0.3, axis='y')

    avg_revision = rev_stats['revision_pct'].abs().mean()
    axes[1].text(0.02, 0.98, f"Avg Revision: {avg_revision:.2f}%",
                 transform=axes[1].transAxes, fontsize=11, verticalalignme
                 bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.7)

plt.tight_layout()
plt.show()

print("\n" + "="*80)
print("CRITICAL POINT: Backtests using final revised data OVERSTATE perfo
print("Always use point-in-time data to get realistic results.")
print("="*80)
```






```
================================================================================
CRITICAL POINT: Backtests using final revised data OVERSTATE performance!
Always use point-in-time data to get realistic results.
================================================================================
```

4.3 Demo: InventoryFairValueModel

Let's build a simple fair value model based on inventory levels.

In [11]:
```
# Create sample dataset for modeling
data = create_sample_dataset(
    commodity='crude_oil',
    n_years=3,
    include_fundamentals=True,
    seed=42
)

print("Sample Dataset:")
print(data.head())
print(f"\nShape: {data.shape}")
print(f"Date range: {data.index[0].date()} to {data.index[-1].date()}")
print(f"\nColumns: {list(data.columns)}")
```

```
Sample Dataset:
                price    volume   inventory  production  consumption  \
date
2022-12-05  70.000000  12190786  400.000000    9.964428    10.491771
2022-12-06  70.541782   4183500  399.031703    9.970821    10.514849
2022-12-07  70.382517   3585534  395.496006    9.960738    10.477455
2022-12-08  71.095013   2128272  386.388523    9.959107    10.469684
2022-12-09  72.794957   4639675  386.655693    9.966061    10.473493

            interest_rate  dollar_index
date
2022-12-05       1.792441    100.148492
2022-12-06       2.016359    100.020974
2022-12-07       2.034898    100.064311
2022-12-08       2.042221    100.003641
2022-12-09       1.995589     99.959868

Shape: (756, 7)
Date range: 2022-12-05 to 2025-10-27

Columns: ['price', 'volume', 'inventory', 'production', 'consumption',
'interest_rate', 'dollar_index']
```

In [12]:

```
# Split data into train and test
split_date = data.index[int(len(data) * 0.8)]

train_data = data[data.index < split_date]
test_data = data[data.index >= split_date]

print(f"Train period: {train_data.index[0].date()} to {train_data.index[-
print(f"Test period: {test_data.index[0].date()} to {test_data.index[-1].
```

```
Train period: 2022-12-05 to 2025-03-27 (604 days)
Test period: 2025-03-28 to 2025-10-27 (152 days)
```

In [13]:

```
# Create and fit inventory-based fair value model

model = InventoryFairValueModel(
    inventory_column='inventory',
    normalize=True,
    include_squared=True,  # Non-linear inventory effect
    regularization=0.1
)

# Prepare features (inventory, interest_rate, dollar_index)
feature_cols = ['inventory', 'interest_rate', 'dollar_index']
X_train = train_data[feature_cols]
y_train = train_data['price']

# Fit model
model.fit(X_train, y_train)

print("\nModel fitted successfully!")
print(f"\nModel parameters:")
params = model.get_params()
print(f"  R²: {model._fit_metadata['r2']:.4f}")
print(f"  Residual std: ${model._residual_std:.2f}")
print(f"  N samples: {model._fit_metadata['n_samples']}")

print(f"\nCoefficients:")
```

```python
for feature, coef in model._coefficients.items():
    print(f"  {feature}: {coef:.4f}")
```

```
Model fitted successfully!

Model parameters:
  R²: 0.3792
  Residual std: $5.47
  N samples: 604

Coefficients:
  inventory: 12.5470
  interest_rate: -1.7219
  dollar_index: -3.7077
  inventory_squared: -11.2073
  intercept: 66.3732
```

In [14]:

```python
# Generate predictions

X_test = test_data[feature_cols]
y_test = test_data['price']

# Point predictions
predictions = model.predict(X_test)

# Probabilistic predictions with confidence intervals
predictions_proba = model.predict_proba(X_test, confidence=0.95)

print("\nPredictions:")
print(predictions_proba.head(10))
```

```
Predictions:
            fair_value      lower      upper
date
2025-03-28   65.770278  55.042661  76.497895
2025-03-31   65.942933  55.215316  76.670550
2025-04-01   66.229838  55.502221  76.957455
2025-04-02   65.348671  54.621054  76.076288
2025-04-03   63.043030  52.315414  73.770647
2025-04-04   63.354740  52.627123  74.082357
2025-04-07   64.556090  53.828473  75.283707
2025-04-08   65.003784  54.276167  75.731401
2025-04-09   65.115846  54.388229  75.843463
2025-04-10   65.145830  54.418213  75.873447
```

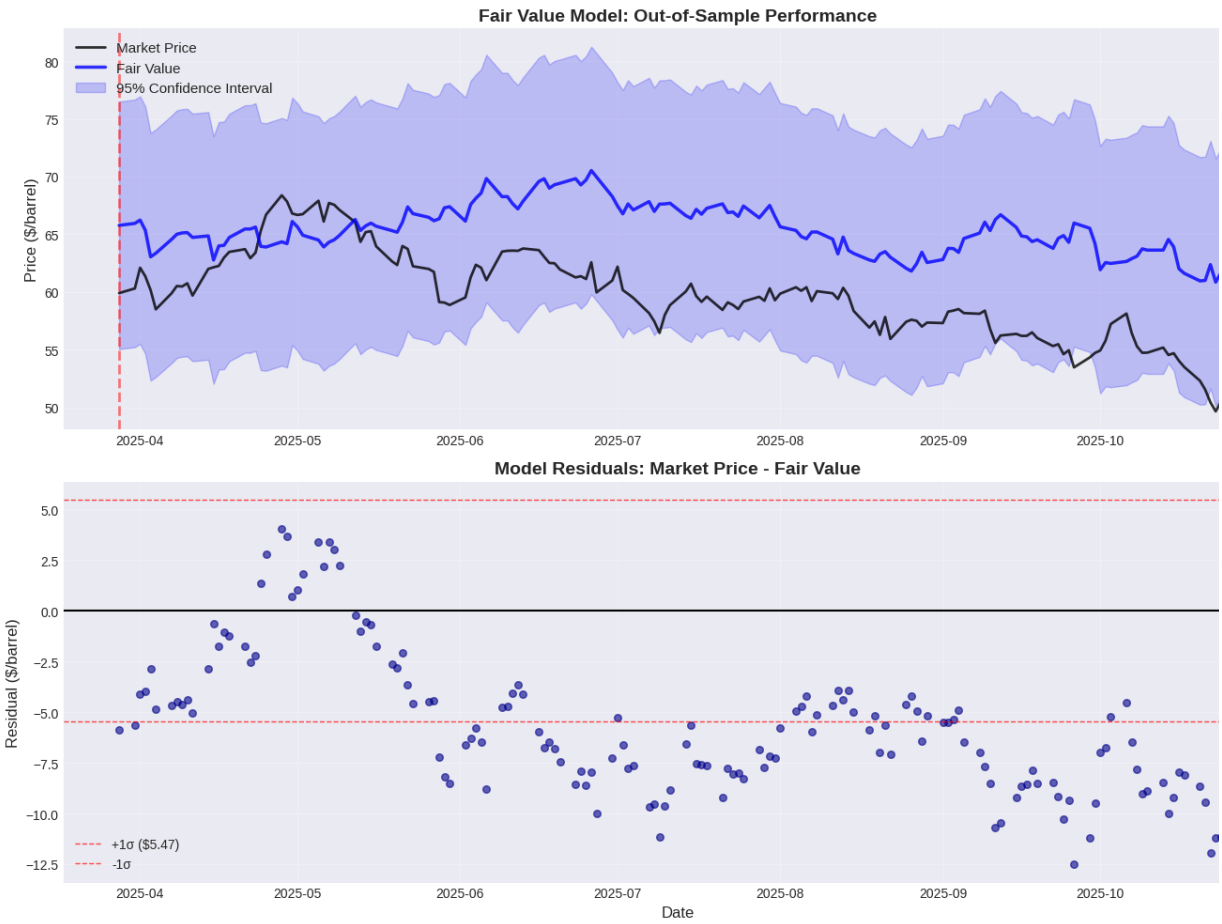
In [15]:

```python
# Evaluate model performance

metrics = model.evaluate(X_test, y_test)

print("\n" + "="*60)
print("MODEL EVALUATION (Out-of-Sample)")
print("="*60)
print(f"MAE: ${metrics['mae']:.2f}")
print(f"RMSE: ${metrics['rmse']:.2f}")
print(f"R²: {metrics['r2']:.4f}")
print(f"Directional Accuracy: {metrics['directional_accuracy']*100:.1f}%"
print(f"Information Coefficient: {metrics['information_coefficient']:.4f}
print(f"IC p-value: {metrics['ic_pvalue']:.4f}")
print(f"N observations: {metrics['n_observations']}")
print("="*60)
```

```
================================================================
MODEL EVALUATION (Out-of-Sample)
================================================================
MAE: $6.09
RMSE: $6.70
R²: -1.9846
Directional Accuracy: 49.0%
Information Coefficient: 0.4507
IC p-value: 0.0000
N observations: 152
================================================================
```

In [16]:

```
# Visualization: Fair value vs market price

fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Panel 1: Price vs Fair Value
axes[0].plot(test_data.index, y_test.values, linewidth=2, color='black',
axes[0].plot(predictions_proba.index, predictions_proba['fair_value'],
            linewidth=2.5, color='blue', label='Fair Value', alpha=0.8)
axes[0].fill_between(predictions_proba.index,
                     predictions_proba['lower'],
                     predictions_proba['upper'],
                     alpha=0.2, color='blue', label='95% Confidence Inter

axes[0].set_ylabel('Price ($/barrel)', fontsize=12)
axes[0].set_title('Fair Value Model: Out-of-Sample Performance', fontsize
axes[0].legend(fontsize=11, loc='upper left')
axes[0].grid(True, alpha=0.3)

# Add train/test divider
axes[0].axvline(x=split_date, color='red', linestyle='--', linewidth=2, a

# Panel 2: Residuals and mispricing
residuals = y_test.values - predictions_proba['fair_value'].values
mispricing = (y_test.values - predictions_proba['fair_value'].values) / p

axes[1].scatter(predictions_proba.index, residuals, alpha=0.6, s=30, colo
axes[1].axhline(y=0, color='black', linestyle='-', linewidth=1.5)
axes[1].axhline(y=model._residual_std, color='red', linestyle='--', linew
axes[1].axhline(y=-model._residual_std, color='red', linestyle='--', line

axes[1].set_xlabel('Date', fontsize=12)
axes[1].set_ylabel('Residual ($/barrel)', fontsize=12)
axes[1].set_title('Model Residuals: Market Price - Fair Value', fontsize=
axes[1].legend(fontsize=10)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"\nResidual Statistics:")
print(f"  Mean: ${residuals.mean():.2f}")
print(f"  Std Dev: ${residuals.std():.2f}")
print(f"  Max absolute: ${np.abs(residuals).max():.2f}")
```




```
Residual Statistics:
  Mean: $-5.70
  Std Dev: $3.52
  Max absolute: $12.51
```

4.4 Understanding Model Components

The fair value model decomposes price into **additive components**:

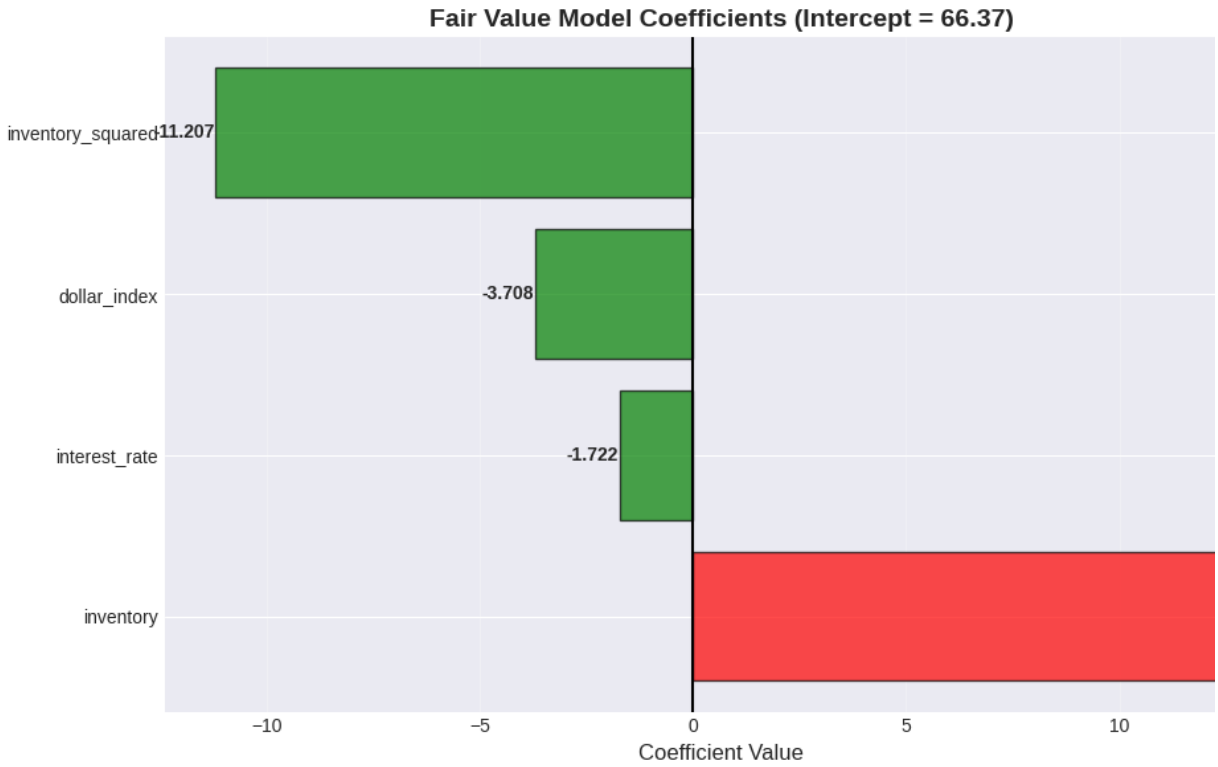
$$ \text{Fair Value} = \beta_0 + \beta_1 \cdot \text{Inventory} + \beta_2 \cdot \text{Int
Rate} + \beta_3 \cdot \text{Dollar Index} + \ldots $$

Each coefficient tells us the **marginal effect** of that variable:

- **Negative inventory coefficient**: Higher inventory → Lower price
- **Positive interest rate coefficient**: Higher rates → Higher price (carry cost)
- **Negative dollar coefficient**: Stronger dollar → Lower commodity price

In [17]:
```
# Visualization: Coefficient importance

coefs = model._coefficients.copy()
intercept = coefs.pop('intercept')

fig, ax = plt.subplots(figsize=(10, 6))

features = list(coefs.keys())
values = list(coefs.values())
colors = ['green' if v < 0 else 'red' for v in values]

bars = ax.barh(features, values, color=colors, alpha=0.7, edgecolor='blac
ax.axvline(x=0, color='black', linestyle='-', linewidth=1.5)
```

```
ax.set_xlabel('Coefficient Value', fontsize=12)
ax.set_title(f'Fair Value Model Coefficients (Intercept = {intercept:.2f}
ax.grid(True, alpha=0.3, axis='x')

# Add value labels
for i, (feature, value) in enumerate(zip(features, values)):
    label_x = value + (0.02 if value > 0 else -0.02)
    ha = 'left' if value > 0 else 'right'
    ax.text(label_x, i, f'{value:.3f}', ha=ha, va='center', fontsize=10,

plt.tight_layout()
plt.show()

print("\nInterpretation:")
print("• Negative coefficients: Higher value → Lower fair value price")
print("• Positive coefficients: Higher value → Higher fair value price")
print("• Magnitude indicates sensitivity (larger = more important)")
```




```
Interpretation:
• Negative coefficients: Higher value → Lower fair value price
• Positive coefficients: Higher value → Higher fair value price
• Magnitude indicates sensitivity (larger = more important)
```

Summary and Key Takeaways

What We Learned

1. **Fair Value Concept**

   - Fair value is a theoretically justified price based on fundamentals
   - Commodities have unique characteristics: physical storage, convenience yield, product economics

    - Gap between market price and fair value creates trading opportunities
2. **Economic Foundations**
    - **Kaldor's Storage Theory**: Futures-spot spread reflects storage costs minus convenier
    - **Working's Hypothesis**: Non-linear relationship between inventory and price
    - **Backwardation vs Contango**: Term structure signals market tightness
3. **Trading Applications**
    - Systematic identification of mispricing opportunities
    - Risk management through confidence intervals
    - Position sizing based on conviction and uncertainty
4. **The Toolkit**
    - `PointInTimeDataFrame`: Prevents look-ahead bias
    - `FairValueModel`: Unified interface for different model types
    - Clean architecture enables backtesting and production deployment

Next Module

Module 2: Point-in-Time Data Architecture

- Deep dive into temporal data management
- Building look-ahead-free backtests
- Handling data revisions and publication lags

Exercises

Exercise 1: Storage Economics

Calculate the implied convenience yield given:

- Spot price: $ 75 /barrel
- 6 -month futures: $ 76 /barrel
- Risk-free rate: 4 % per annum
- Storage cost: 3 % per annum

Is the market in contango or backwardation?

Exercise 2: Model Comparison

Create a `SupplyDemandModel` using production and consumption columns. Compare its perf to the `InventoryFairValueModel` on the test set.

Exercise 3: Trading Strategy

Develop a simple trading strategy that:

1. Goes long when mispricing < - 5 %
2. Goes short when mispricing > + 5 %

3 . Exits when price returns to within ± 2 % of fair value

Calculate the strategy's Sharpe ratio on the test set.

End of Module 1